

FastAPI Learn Tutorial - User Guide

Tutorial - User Guide

This tutorial shows you how to use **FastAPI** with most of its features, step by step.

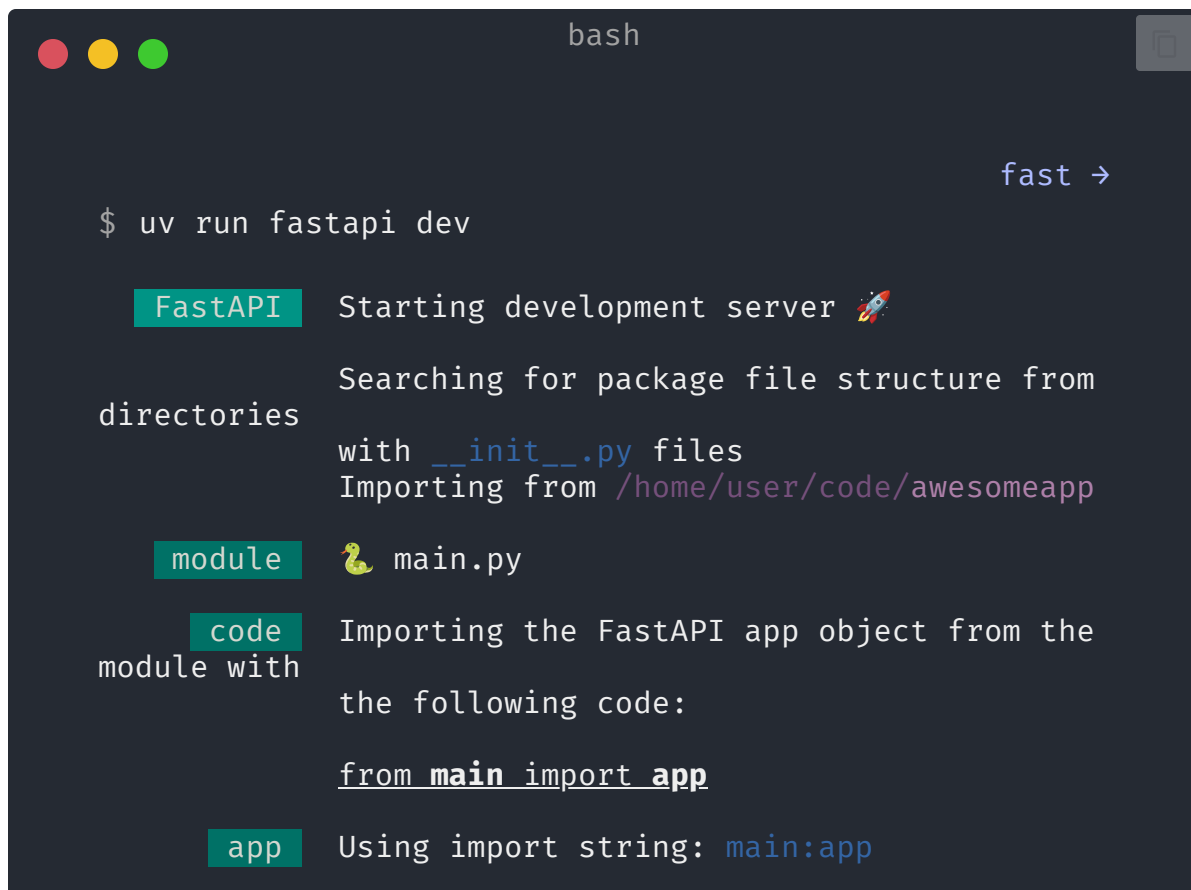
Each section gradually builds on the previous ones, but it's structured to separate topics, so that you can go directly to any specific one to solve your specific API needs.

It is also built to work as a future reference so you can come back and see exactly what you need.

Run the code

All the code blocks can be copied and used directly (they are actually tested Python files).

To run any of the examples, copy the code to a file `main.py`, and start `fastapi dev` with `uv run`:



```
bash
fast →
$ uv run fastapi dev
FastAPI Starting development server 🚀
Searching for package file structure from
directories with __init__.py files
Importing from /home/user/code/awesomeapp
module 🦹 main.py
code Importing the FastAPI app object from the
module with the following code:
from main import app
app Using import string: main:app
```

```
server Server started at http://127.0.0.1:8000
server Documentation at
http://127.0.0.1:8000/docs

tip Running in development mode, for
production use:
    fastapi run

Logs:

INFO Will watch for changes in these
directories:
    ['/home/user/code/awesomeapp']
INFO Uvicorn running on http://127.0.0.1:8000
(Press CTRL+C
to quit)
INFO Started reloader process [383138] using
WatchFiles
INFO Started server process [383153]
INFO Waiting for application startup.
INFO Application startup complete.
```

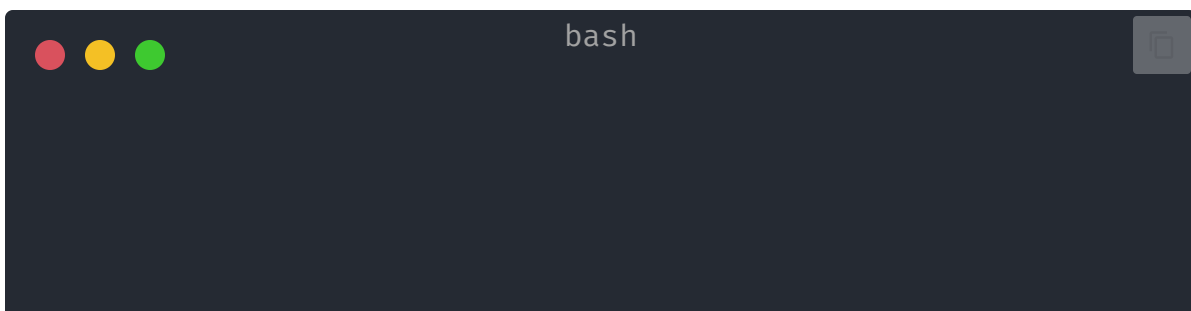
It is **HIGHLY encouraged** that you write or copy the code, edit it and run it locally.

Using it in your editor is what really shows you the benefits of FastAPI, seeing how little code you have to write, all the type checks, autocompletion, etc.

Install FastAPI

The first step is to set up your project and add FastAPI.

Install [uv](#), then create a project and add FastAPI:



`uv add` creates the project's virtual environment in `.venv`, adds FastAPI to `pyproject.toml`, and creates `uv.lock` so the same package versions can be installed later.

What these commands do

- `uv init`: create a new Python project.
- `awesome-project`: create the project in a new directory with this name.
- `--bare`: create only the minimal `pyproject.toml` file, without generating a sample `main.py`, `README.md`, or other files. You will create the application files yourself in the next steps of this tutorial.

Then `cd awesome-project` enters the new project directory before adding FastAPI.

`uv` will use a compatible Python version already installed on your system, or download one if needed.

When you run `uv add`, it selects compatible versions of FastAPI and all the packages FastAPI depends on. It records the exact versions in `uv.lock`, making it possible to install the same package versions later on another computer or when deploying the application.

Creating or updating this file is called **locking the project dependencies** [↗](#). `uv` does this automatically when you add a package.

FastAPI installation options

When you install with `uv add "fastapi[standard]"` it comes with some default optional standard dependencies, including `fastapi-cloud-cli`, which allows you to deploy to [FastAPI Cloud](#) [↗](#).

If you don't want to have those optional dependencies, you can instead install `uv add fastapi`.

If you want to install the standard dependencies but without the `fastapi-cloud-cli`, you can install with `uv add "fastapi[standard-no-fastapi-cloud-cli]"`.

Using `pip` instead

If you prefer to manage a virtual environment and packages manually, create and activate a virtual environment and then install FastAPI with `pip install "fastapi[standard]"`.

Read the [Virtual Environments guide](#) for the detailed steps.

AI Agent Skills

FastAPI includes an official skill for AI coding agents. It is bundled with the package, so its guidance stays aligned with the version of FastAPI installed in your project and updates when you update FastAPI.

After installing FastAPI in your project, you can install the skill with [Library Skills](#):

```
uvx library-skills
```

Note

`uvx` is an alias for `uv tool run`. It runs Library Skills in a temporary, isolated environment while Library Skills scans the packages installed in your project.

The skill is compatible with Codex, Claude Code, Cursor, GitHub Copilot, Gemini CLI, Pi, OpenCode, and most other coding agents. For Claude Code, select `.claude/skills` when asked where to install the skill.

Advanced User Guide

There is also an **Advanced User Guide** that you can read later after this **Tutorial - User Guide**.

The **Advanced User Guide** builds on this one, uses the same concepts, and teaches you some extra features.

But you should first read the **Tutorial - User Guide** (what you are reading right now).

It's designed so that you can build a complete application with just the **Tutorial - User Guide**, and then extend it in different ways, depending on your needs, using some of the additional ideas from the **Advanced User Guide**.